

Agent 平替 v0.1 —— 短名单 + 路由建议

- 你在看：v0.1 结果报告 —— 套上达标线后的短名单 + per-task 路由 + 混合省钱率
- 上游：v0 结果报告 —— 质量 × 成本表（跑完之后的结果）
- 底表：per-task 矩阵 CSV —— 18×4（score + token + \$），本报告所有数字的来源
- 证据：run-log · bug 写报：findings
- 达标线：Paul 锁定 质量 ≥ 70% / 成本 ≤ ½（起步值，见 DECISIONS）
- 口径：common-14 公共题集，与 v0 主表一致、跨模型可比
- 数字：均已对底表逐项重算、可回溯

0. 结论

在 Paul 锁定的达标线（质量 ≥ 70% / 成本 ≤ ½）下，便宜模型短名单 = 仅 GLM-4.6。但真正的产品结论不在「换不换 GLM」，而在怎么路由：

- 纯换 GLM（不路由）= 76% 质量 / 省 8.4x；
- 按题路由（难题回退 Claude）= 99% 质量 / 省 ~2x。

后者几乎不丢质量、成本仍砍一半，并且落在 Paul 设的 ≤½ 线上——这就是 hybrid 路由要交付的甜区，也是 v0.1 把「能不能替」推进到「怎么替最划算」的一步。

1. 短名单：套达标线（common-14）

模型	质量	=Claude	\$/题	=Claude 成本	便宜	过线?
Claude sonnet-4.6	0.653	100%	\$0.439	1.00	1x	基准
GLM-4.6	0.497	76%	\$0.052	0.12	8.4x	✅ 进短名单
DeepSeek V3.2	0.433	66%	\$0.162	0.37	2.7x	❌ 质量不过
Qwen3-Coder	0.377	58%	\$0.209	0.48	2.1x	❌ 质量不过

质量是唯一筛子。DeepSeek / Qwen 的成本其实都已 ≤½，被卡的全在 70% 质量线——所以这条线下短名单干净（只有 GLM），对外我们的话术是：纯按质量筛掉，没有靠成本门槛硬卡。

2. Per-task 分档：14 题里 10 道走 GLM、4 道留 Claude

我们把 14 道题逐题对比 GLM 与 Claude，门槛设为「该题 GLM 得分 ≥ Claude 的 70%」。结果清楚地分成两类：

- 🟢 10 道 GLM-safe。GLM 在这些题上达到甚至超过门槛，可以直接交给便宜模型——其中 financial_stmt、pe_screening、os_log、tcga 这几道 GLM 多数拿到了 100% 甚至更高，另外 k8s、

homework、agora 等也都过线。

- 4 道 Claude-only。GLM 在这些题上明显做不出来，必须保留 Claude: causal_ihdp_ite_estimation (Claude 0.178 / GLM 0.000)、crf_sdtm_mapping (0.589 / 0.000)、molecular_structure_plausibility (0.500 / 0.167)、capacitated_vehicle_routing (1.000 / 0.000)。

这说明路由的正确形态不是「整体把 Claude 换成 GLM」，而是「按题分流」：能交给 GLM 的交给 GLM、难题回退 Claude。上面这 10 比 4 的切分，就是路由策略表最初的雏形。

3. 混合省钱曲线★（核心结论）

策略	质量	=Claude	\$/题	便宜
全 Claude（基准）	0.653	100%	\$0.439	1×
纯 GLM（不路由）	0.497	76%	\$0.052	8.4×
Per-task 混合 / verify 兜底	0.647	99%	~\$0.22	~2×

一句话：纯换 GLM 丢掉 24% 质量换 8.4× 便宜；按题路由只丢 1% 质量，仍省 ~2×——难题回退 Claude，几乎不丢质量、成本砍半。而且这个混合点正落在 Paul 设的 $\leq \frac{1}{2}$ 成本线上 (0.5× cost / 99% 质量)，说明他这条起步线在 per-task 混合下是可达的甜区，不是空设。

路由机制：便宜模型先跑 → 自动判分 → 没过回退 Claude (verify-then-route)。质量与 oracle 分流持平 (0.647)，成本略高 (\$0.22，因为回退题付了两遍)。

4. 局限

- 「99% / 2×」是可达上界，不是生产实测。它是在 ALE 14 题上、用真实判分器模拟分流得到的；生产里没有现成判分器，实际落点取决于「分类器 + verify 代理」的准确度——会落在「纯 GLM 76%」与「上界 99%」之间。
- 样本小：common-14；趋势可信，绝对值待更大 N 收敛。
- 边角：epidemiology_forecast 两个模型都 0 分，被归入 GLM-safe（都失败→走便宜不丢质量），属正常处理、已知悉。
- 分布：仍是 ALE 公开基准，≠ BFC 真实流量；最终要真实流量数据接力。

5. 下一步（谁做什么）

SDE (Abby) —— 把底表变成路由器。下面每个要运行 / 打开的文件都标了路径（相对 repo 根目录）；脚本只用 Python 标准库、无需安装依赖，统一在 repo 根目录下运行。起手先开 analysis/README.md。

第一步 · 复现并确认对齐（可立即开工，不依赖达标线）

- 在 repo 根目录运行：python3 analysis/apply_threshold.py

- 它读入：`reports/v0-per-task-matrix.csv` (18x4 底表，已提交)
- 它产出：终端打印第 1 节短名单、第 2 节分档、第 3 节省钱曲线；并写出 `reports/routing-policy.csv` (每道题 → 走 GLM 还是 Claude)
- 跑出来该是什么 (对齐判据)：短名单 GLM 76% / 8.4x、DeepSeek 与 Qwen 都卡质量；分档 10 道 GLM-safe / 4 道 Claude-only；曲线 纯 GLM 76% / 8.4x、按题混合 99% / ~2x。和本报告一致即对齐。
- 想换达标线：改 `analysis/apply_threshold.py` 顶部的 `Q_BAR` / `C_BAR` 两个常量再跑。
- 想弄清这些数怎么来的：看 `analysis/README.md` (脚本对应哪一节、用了哪些单价)，以及 `analysis/extract_matrix.py` (底表的来源；注意它读的原始 `.logs/` 不在 repo、只在本地)。
- 做完的标志：你这边跑出的数和报告一致，且生成了 `reports/routing-policy.csv`。

第二步 · verify-then-route 原型 (兜底层，不依赖达标线，分两段)

「自动判分用什么来判」必须先分清，否则会卡住：

- ① 离线段 (先做，纯数据、不用真跑模型)：模拟「GLM 先跑、没过就回退 Claude」，在这 14 题上复现 99% / 2x 那条线。逻辑已经在 `analysis/apply_threshold.py` 里——具体是 `blended()` 函数，以及名为 `Verify-then-route (realistic)` 的那条 policy，照着改即可。做完的标志=一个能跑出 99% / 2x 的原型。
- ② 生产段：生产里没有 ALE 那个判分器，所以要设计一个 accept/reject 代理 (judge 模型或启发式规则) 来替代它，决定「这次 GLM 的输出该不该被接受」。验证时拿 `reports/routing-policy.csv` 里的 `route_to` 列当对照。做完的标志=代理方案 + 它在 14 题上判「该不该回退」的准确率。

第三步 · 轻量分类器 (先做设计、不训模型)

- 把用户 prompt 判成任务类型 + 难度。标签空间 = ALE 的 8 个领域，可直接从 `reports/v0-per-task-matrix.csv` 的 `task` 列前缀读出来：business_finance / computing_math / health_medicine / life_sciences / legal / physical_sciences / education_info / transport_safety。
- 带置信度，置信度低时默认把任务留给 Claude，避免判错就路由错。
- 训练数据很薄 (只有这 14 道、8 个领域各一两例)，所以现阶段是定方案、定标签空间、定置信门槛，不是训模型。做完的标志=一页设计 + 标签空间 + 低置信兜底规则。

第四步 · 真实流量阶段再接力

- 用各模型官方计费价核对我们自算的成本 (现用价目见 `analysis/README.md` 的价格表)、从 token 或渠道反推任务类型、接入线上的成败信号与打点字段，把「公开基准离线分级」过渡到「自有流量在线细化」。

起手两步： 先打开 `analysis/README.md` 读一遍，再在 repo 根目录跑 `python3 analysis/apply_threshold.py`。

Elena (PM)： 负责本报告与套表的交付 (短名单加路由建议)

Paul： $\leq \frac{1}{2}$ 是一个起步值。等 sde 把 verify-then-route 的真实落点跑出来、或者接上真实流量数据之后，再决定这条线是否需要调整。

本阶段不做： 不扩 N、不加 Windows/GPU 题、不重测 Kimi (见 [DECISIONS](#))。

数据与全程证据：repo `agent-eval-v0` (底表 CSV / run-log / findings)。达标线决策见 [DECISIONS](#)。

Repo: <https://github.com/yingshill/agent-eval-v0>